07: Time Series – Classical Methods

When the order of the rows is the whole point

Every model so far assumed the rows were interchangeable

Regression, trees, logistic regression – all of them treat the dataset as a *bag* of i.i.d. rows. Shuffle the rows, nothing changes.

Time series breaks that assumption. Row order *is* the signal. Yesterday's value tells you a lot about today's. You can only train on the past, and you must predict the future – never the other way around.

Examples everywhere: daily sales, electricity demand, website traffic, AMD / Dram exchange rate, sensor readings, COVID case counts, your heart rate.

Two lectures. **Today (07):** the classical statistical toolkit – decomposition, stationarity, ARIMA, exponential smoothing. **Next (08):** treating forecasting as a supervised ML problem.

Outline

What makes time series different

One column, indexed by time – and the index carries information.

The vocabulary

A time series is a sequence $y_1, y_2, \ldots, y_T$ sampled at regular steps (hourly, daily, monthly, ...).

Two jobs:

- **Forecasting** – predict future values $y_{T+1}, \ldots, y_{T+h}$. (h = forecast *horizon*.)
- **Understanding** – decompose, explain, find structure.

Two shapes of data:

- **Univariate** – one series.
- **Multivariate** – many series, possibly with exogenous drivers.

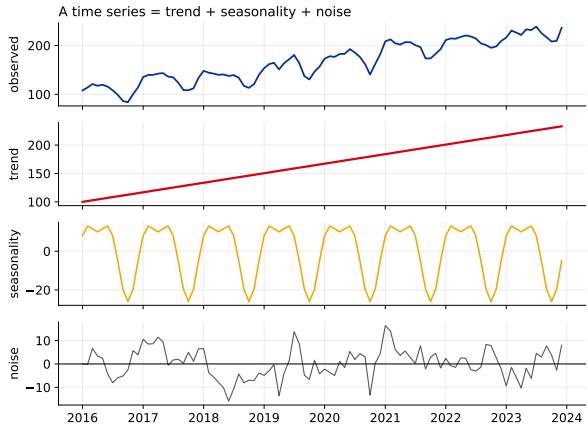
The one rule that governs everything: information flows forward in time only. Any value, feature, or statistic you feed the model must have been *knowable at prediction time*. Violate this and you get "leakage" – great backtests, useless in production.

The four components of a series

A series is usefully thought of as a sum (or product) of:

- **Trend** T_t – long-run drift up or down.
- **Seasonality** S_t – a pattern that repeats on a *fixed* period (24h, 7d, 12mo).
- **Cycle** – swings with no fixed period (business cycles).
- **Noise / residual** R_t – what's left.

Additive: $y_t = T_t + S_t + R_t$.
Multiplicative: $y_t = T_t \cdot S_t \cdot R_t$ (season grows with level – take log to make it additive).



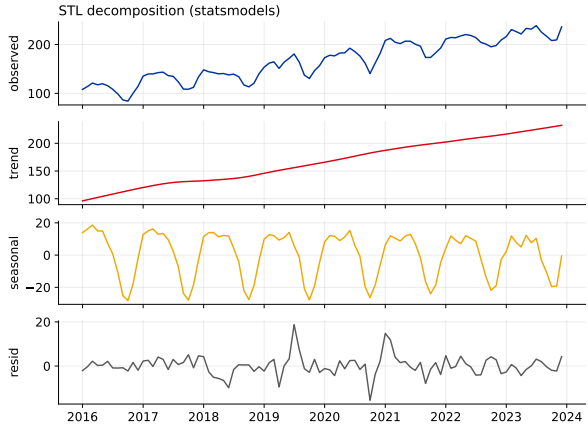
Decomposition: STL

STL = "Seasonal-Trend decomposition using Loess". It splits the series into trend + seasonal + residual, robustly.

Why bother?

- *See* the structure before modeling.
- Deseasonalize (analyze the trend cleanly).
- The residual is where anomalies show up.

```
from statsmodels.tsa.seasonal
    import STL
res = STL(y, period=12).fit()
res.trend, res.seasonal, res.
    resid
```



Stationarity and differencing

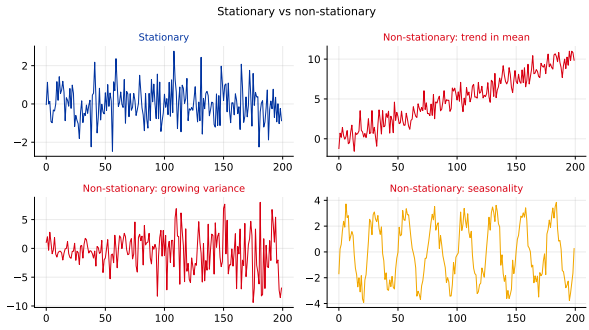
The classical models need a series whose behaviour does not drift.

What is stationarity?

A series is (weakly) **stationary** if its statistical behaviour does not depend on *when* you look:

- constant mean (no trend),
- constant variance (no growing swings),
- autocovariance depends only on the *lag*, not on absolute time.

Why we care: ARMA-type models assume stationarity. Trends and seasonality must be removed *first*, then added back to the forecast.



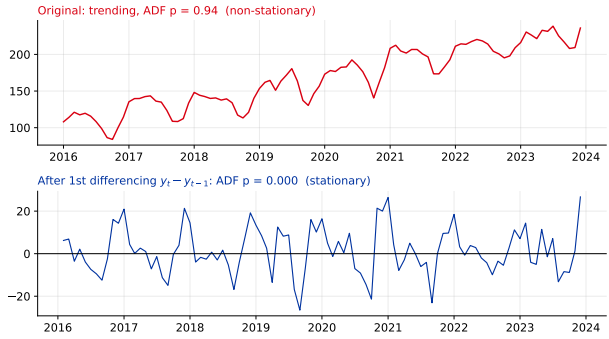
Differencing makes a series stationary

Differencing: model the change instead of the level.

$$y'_t = y_t - y_{t-1}$$

- Removes a trend (do it d times – usually $d = 1$).
- **Seasonal differencing** $y_t - y_{t-m}$ removes a season of period m.

Testing it – the ADF test.
Augmented Dickey-Fuller: $H_0 =$ "non-stationary". *Small p-value* $\Rightarrow$ reject $\Rightarrow$ stationary.



A trap: differencing too many times injects artificial noise. Difference the *minimum* amount that passes the test.

Autocorrelation

How a series correlates with delayed copies of itself.

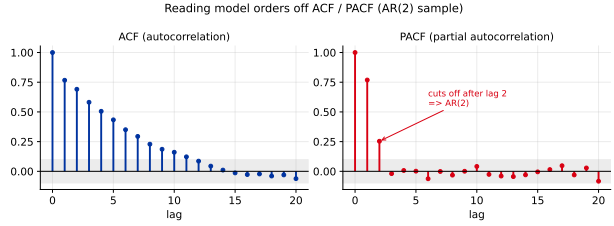
ACF and PACF

ACF (autocorrelation): correlation between y_t and y_{t-k} for each lag k. Includes indirect effects.

PACF (partial): correlation at lag k *after* removing the effect of the shorter lags. The direct link only.

They are the classic way to read off model orders:

- PACF cuts off after lag $p \Rightarrow$ **AR(p)**.
- ACF cuts off after lag $q \Rightarrow$ **MA(q)**.
- Both decay $\Rightarrow$ mix (ARMA).



The shaded band is the "not significantly different from zero" region ($\pm 1.96/\sqrt{T}$).

The ARIMA family

The workhorse of classical forecasting.

Building up: AR, MA, ARMA

AR(p) – autoregressive. Regress on your own past:

$$y_t = c + \sum_{i=1}^{p} \phi_i \, y_{t-i} + \varepsilon_t$$

"Tomorrow looks like a weighted blend of recent days."

MA(q) – moving average. Regress on past *shocks*:

$$y_t = c + \varepsilon_t + \sum_{j=1}^{q} \theta_j \, \varepsilon_{t-j}$$

"A surprise last week still echoes this week."

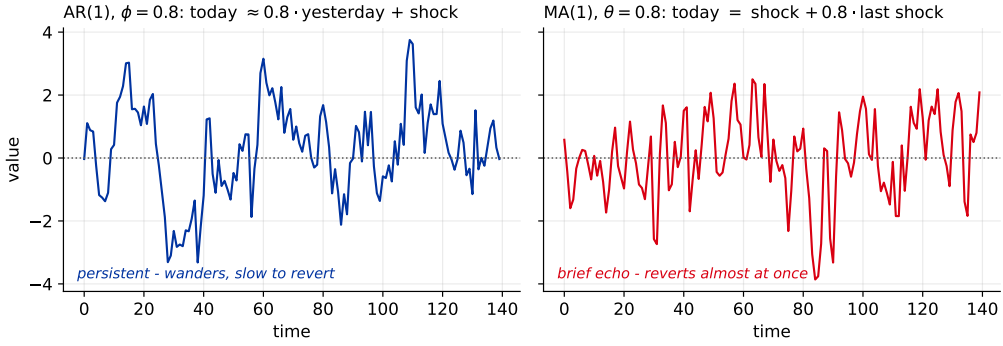
ARMA(p, q) combines both – for a *stationary* series.

ARIMA(p, d, q) adds the "I" = Integrated = difference d times first, so it handles trending series directly.

- p – AR order (from PACF)
- d – differencing (from ADF)
- q – MA order (from ACF)

ARIMA(0,1,0) is just the random walk – the "tomorrow = today" baseline.

What AR and MA actually look like



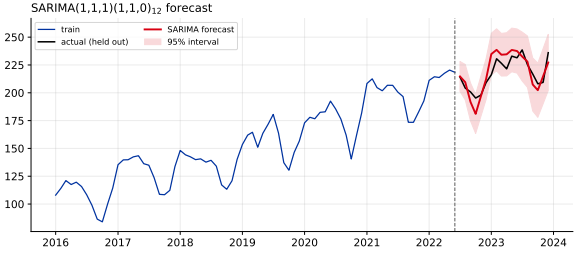
Same machinery, different memory: an **AR** term makes the series *persist* (today leans on yesterday's *value*); an **MA** term is a short *echo* of the last random *shock*. ARMA blends the two.

SARIMA: adding seasonality

Real series are seasonal, so add a second, seasonal triple:

$$\text{SARIMA}(p, d, q)(P, D, Q)_m$$

- lower-case (p, d, q) – the short-term, non-seasonal part;
- upper-case (P, D, Q) – the same idea at the seasonal lag;
- m – periods per season (12 monthly, 7 daily-with-weekly, 24 hourly-with-daily).



The forecast comes with a **prediction interval** – uncertainty grows with the horizon. A point forecast without an interval is only half an answer.

In practice: don't hand-tune p, d, q. Use `pmdarima.auto_arima` or `statsforecast.AutoARIMA`, which search orders by AIC.

Exponential smoothing (ETS)

Weighted averages where the recent past counts most.

From a simple average to Holt-Winters

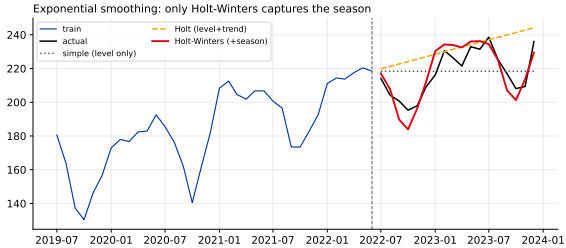
Simple (SES) – level only. Weights decay geometrically into the past:

$$\hat{y}_{t+1} = \alpha y_t + (1-\alpha)\hat{y}_t$$

Flat forecast – no good if there is a trend or season.

Holt – adds a trend term (level + slope). Forecast is a line.

Holt-Winters – adds a seasonal term. Level + trend + season, each with its own smoothing rate (α, β, γ).



ETS and ARIMA overlap but are not the same: ETS is built from interpretable components (level/trend/season); ARIMA is built from autocorrelation. On many business series they perform similarly – try both.

Baselines and evaluation

A forecast is only "good" relative to the dumbest thing that works.

Always start with a naive baseline

Before any model, write down the trivial forecast:

- **Naive:** $\hat{y}_t = y_{t-1}$ (tomorrow = today).
- **Seasonal naive:** $\hat{y}_t = y_{t-m}$ (this December = last December).
- **Drift:** last value + average change.

If your fancy model can't beat seasonal naive, it is not helping. This happens far more often than people admit.

Error metrics

- **MAE** – mean |error|, in the target's units.
- **RMSE** – penalizes big misses more.
- **MAPE** – percentage error; breaks when $y \approx 0$ and is asymmetric.
- **MASE** = MAE / $\text{MAE}_{\text{seas. naive}}$ – error relative to the naive forecast. < 1 beats naive, > 1 loses. Scale-free; the one to report.

Evaluate on the future, never by random split

You cannot shuffle-and-split a time series: that trains on the future to predict the past. Instead use a **time-ordered holdout** and **backtesting** (rolling / expanding windows).

This "time-aware validation" is the hinge between the two lectures – it is exactly what lets us hand forecasting to a generic ML model next time. We build the full machinery (leakage, lag features, `TimeSeriesSplit`) in lecture 08.

Rule of thumb for picking an approach: short, single, strongly-seasonal series → ARIMA / ETS are hard to beat. Many related series, long history, exogenous drivers → the ML and deep methods of the next lecture start to win.

Recap – classical time series

Ideas that transfer:

- Time order is information; only train on the past.
- Decompose into trend + season + residual (STL).
- Make it stationary (difference; check with ADF).
- ACF/PACF read off AR/MA orders.
- ARIMA / SARIMA: autocorrelation-based.
- Exponential smoothing: component-based.
- Score against seasonal naive with MASE; evaluate on the future.

Next (08): ML for time series. Reframe forecasting as supervised learning –

- time-aware splits and leakage,
- lag / rolling / calendar features,
- `TimeSeriesSplit` cross-validation,
- gradient boosting (and the trend-extrapolation trap),
- deep learning and the new foundation models.

Questions?

Next: forecasting as a supervised ML problem.